SCR2043 OPERATING SYSTEMS

Name	:	MUHAMMAD AFIQ DANIAL BIN ROZAIDIE	Marks
Student ID	:	A23CS0117	
Section	:	02	

This lab assessment is designed to test your understanding and skills on some basic concepts and tools related to process monitoring and management in operating system. Please follow the instructions carefully and submit your answers in this word document and rename the file as **os-lab-assessment02-studentname-matricno.docx**.

Essential Steps Before Starting Lab Assessment 2:

1. **Download necessary source codes:**

   Use the `wget` command to retrieve the following source code files to your Linux (or WSL or MacOS) environment:

```
wget -O mainprocess.c https://rebrand.ly/mainprocess_c
wget -O subprocess1.c https://rebrand.ly/subprocess1_c
wget -O subprocess2.c https://rebrand.ly/subprocess2_c
```

2. **Compile the source files:**

   Use the `gcc` compiler to create executable files from the source code.

```
gcc mainprocess.c -o mainprocess
gcc subprocess1.c -o subprocess1
gcc subprocess2.c -o subprocess2
```

3. **Execute the dummy processes:**

   Run all the dummy processes

```
./mainprocess &
```

   Press **enter** two times.

4. The dummy processes are running for 2 hours. If you took longer than 2 hours on questions 1-9, please restart the main process with `./mainprocess &`.

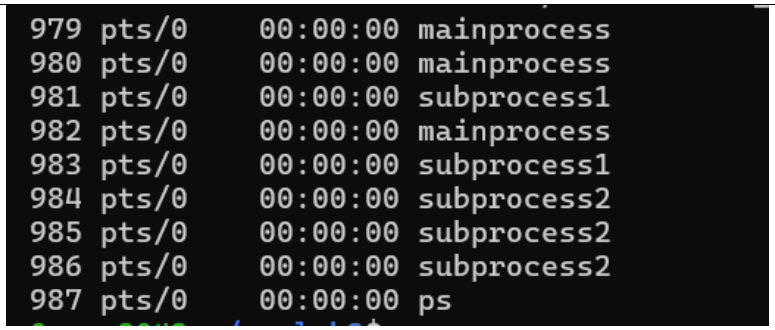
Lab Assessment 2 : Linux Process Monitoring and Management

Instructions:

1. Carefully execute each command as instructed in the questions.
2. Write down the exact command used for each task.
3. Capture a screenshot of the command's output.

Question 1

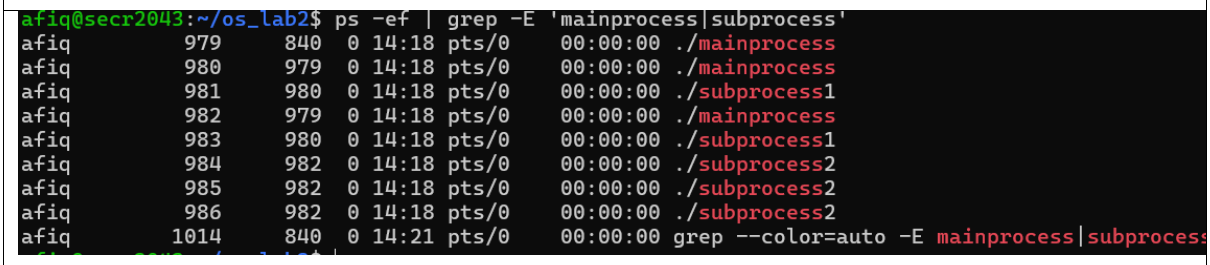
Use the `ps` command with the appropriate option to display a complete list of all running processes within the Linux operating system.

Command
ps -e
Output


```
979 pts/0    00:00:00 mainprocess
980 pts/0    00:00:00 mainprocess
981 pts/0    00:00:00 subprocess1
982 pts/0    00:00:00 mainprocess
983 pts/0    00:00:00 subprocess1
984 pts/0    00:00:00 subprocess2
985 pts/0    00:00:00 subprocess2
986 pts/0    00:00:00 subprocess2
987 pts/0    00:00:00 ps
```

Question 2

Employ the `ps` command with necessary options to unveil comprehensive details about each running process.

Command
ps -ef | grep -E 'mainprocess|subprocess'
Output


```
afiq@secr2043:~/os_lab2$ ps -ef | grep -E 'mainprocess|subprocess'
afiq         979     840  0 14:18 pts/0    00:00:00 ./mainprocess
afiq         980     979  0 14:18 pts/0    00:00:00 ./mainprocess
afiq         981     980  0 14:18 pts/0    00:00:00 ./subprocess1
afiq         982     979  0 14:18 pts/0    00:00:00 ./mainprocess
afiq         983     980  0 14:18 pts/0    00:00:00 ./subprocess1
afiq         984     982  0 14:18 pts/0    00:00:00 ./subprocess2
afiq         985     982  0 14:18 pts/0    00:00:00 ./subprocess2
afiq         986     982  0 14:18 pts/0    00:00:00 ./subprocess2
afiq        1014     840  0 14:21 pts/0    00:00:00 grep --color=auto -E mainprocess|subprocess
```

Question 3

Use the `ps` command with some tools to only list processes named "subprocess" and show some info about them.

Command
`ps -ef | grep subprocess`
Output

```
afiq@secr2043:~/os_lab2$ ps -ef | grep -E 'subprocess'
afiq         981     980  0 14:18 pts/0    00:00:00 ./subprocess1
afiq         983     980  0 14:18 pts/0    00:00:00 ./subprocess1
afiq         984     982  0 14:18 pts/0    00:00:00 ./subprocess2
afiq         985     982  0 14:18 pts/0    00:00:00 ./subprocess2
afiq         986     982  0 14:18 pts/0    00:00:00 ./subprocess2
afiq        1017     840  0 14:22 pts/0    00:00:00 grep --color=auto -E subprocess
afiq@secr2043:~/os_lab2$
```

Question 4

Execute the `ps` command, specifying options that reveal only the following columns:

- Process ID (`pid`)
- Owner of the process (`user`)
- CPU percentage (`pcpu`)
- Memory percentage (`pmem`)
- Command (`cmd`)

Command
`ps -eo pid,user,pcpu,pmem,cmd | grep -E 'subprocess'`
Output

```
afiq@secr2043:~/os_lab2$ ps -eo pid,user,pcpu,pmem,cmd | grep -E 'subprocess'
    981 afiq      0.0  0.1 ./subprocess1
    983 afiq      0.0  0.1 ./subprocess1
    984 afiq      0.0  0.1 ./subprocess2
    985 afiq      0.0  0.1 ./subprocess2
    986 afiq      0.0  0.1 ./subprocess2
   1059 afiq      0.0  0.1 grep --color=auto -E subprocess
```

Question 5

Building on the ps command used in Question 4, can you add an option to sort the listed processes by their memory usage (`pmem`)?

Command
`ps -eo pid,user,pcpu,pmem,cmd –sort=pmem | grep -E 'subprocess'`
Output

```
afiq@secr2043:~/os_lab2$ ps -eo pid,user,pcpu,pmem,cmd --sort=pmem | grep -E 'subprocess'
    985 afiq      0.0  0.1 ./subprocess2
    981 afiq      0.0  0.1 ./subprocess1
    983 afiq      0.0  0.1 ./subprocess1
    984 afiq      0.0  0.1 ./subprocess2
    986 afiq      0.0  0.1 ./subprocess2
   1066 afiq      0.0  0.1 grep --color=auto -E subprocess
```

Question 6

Construct a command using `ps`, suitable options, and any additional tools to visualize the hierarchical structure (tree-like) of the following processes:

- "mainprocess"
- "subprocess1"
- "subprocess2"

Command
ps –forest -C mainprocess -C subprocess1 -C subprocess2
Output

```
afiq@secr2043:~/os_lab2$ ps --forest -C mainprocess -C subprocess1 -C subprocess2
    PID TTY          TIME CMD
    979 pts/0    00:00:00 mainprocess
    980 pts/0    00:00:00  \_ mainprocess
    981 pts/0    00:00:00  |   \_ subprocess1
    983 pts/0    00:00:00  |   \_ subprocess1
    982 pts/0    00:00:00  \_ mainprocess
    984 pts/0    00:00:00      \_ subprocess2
    985 pts/0    00:00:00      \_ subprocess2
    986 pts/0    00:00:00      \_ subprocess2
```

Question 7

Use `pstree` command with option that show the number of threads to each process.

Command
pstree -c -s 979
Output

```
afiq@secr2043:~/os_lab2$ pstree -c -s 979
systemd---sshd---sshd---sshd---bash---mainprocess-+-mainprocess-+-subprocess1
                                                  |             `-subprocess1
                                                  `-mainprocess-+-subprocess2
                                                                |-subprocess2
                                                                `-subprocess2
```

Question 8

Use `renice` command to change priority level of one of process “subprocess1”.

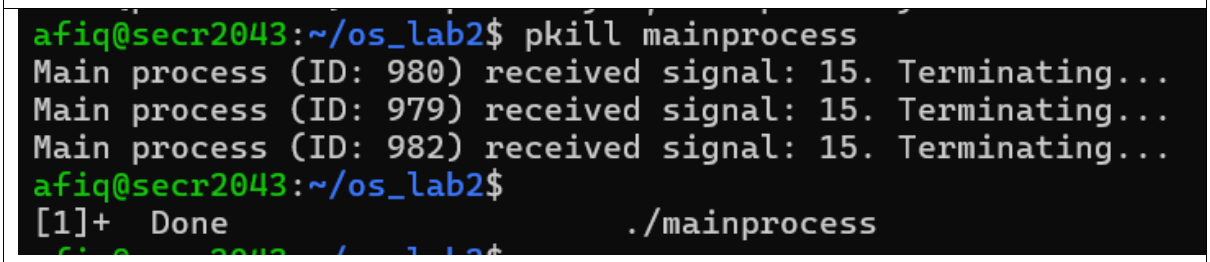
Command
sudo renice -n -10 -p 981
Output

```
afiq@secr2043:~/os_lab2$ ps -o pid,nice,comm -C subprocess1
    PID  NI COMMAND
    981   0 subprocess1
    983   0 subprocess1
afiq@secr2043:~/os_lab2$ sudo renice -n -10 -p 981
981 (process ID) old priority 0, new priority -10
```

Question 9

Terminate all running processes with the name “mainprocess”.

Command
pkill mainprocess
Output



Question 10

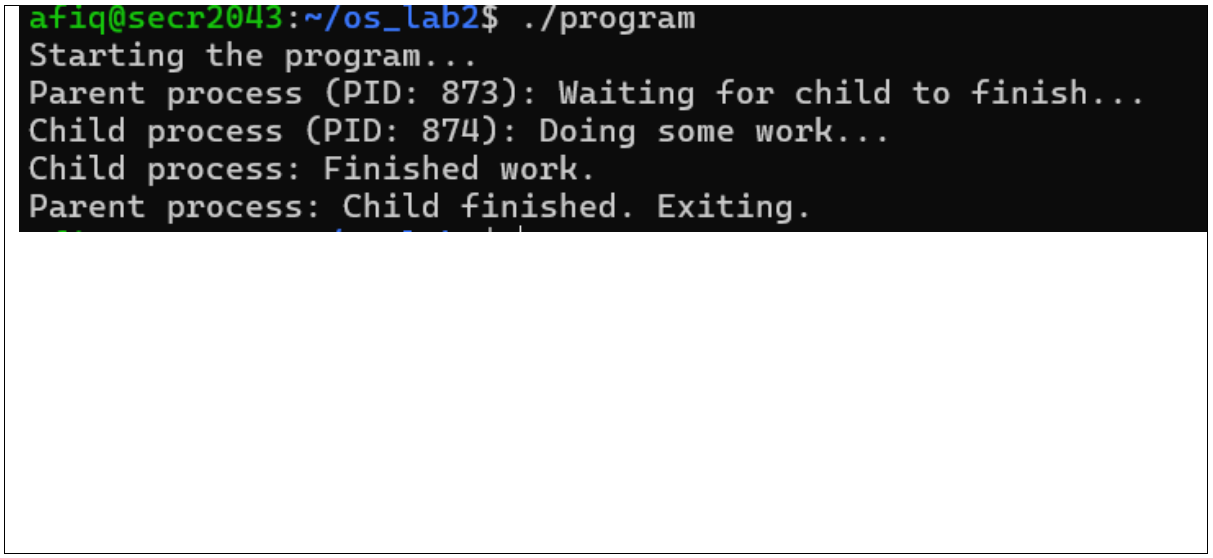
Write a short C or Python code (choose only one language) demonstrating multiprocessing with `fork()` and `wait()`. Compile and/or run the code. Show the output.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    printf("Starting the program...\n");

    pid = fork();  // Create a new process

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    } else if (pid == 0) {
        // Child process
        printf("Child process (PID: %d): Doing some work...\n", getpid());
        sleep(2);
        printf("Child process: Finished work.\n");
    } else {
        // Parent process
        printf("Parent process (PID: %d): Waiting for child to finish...\n", getpid());
        wait(NULL);  // Wait for the child process to finish
        printf("Parent process: Child finished. Exiting.\n");
    }

    return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    printf("Starting the program...\n");

    pid = fork();  // Create a new process

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    } else if (pid == 0) {
        // Child process
        printf("Child process (PID: %d): Doing some work...\n", getpid());
        sleep(2);
        printf("Child process: Finished work.\n");
    } else {
        // Parent process
        printf("Parent process (PID: %d): Waiting for child to finish...\n", getpid());
        wait(NULL);  // Wait for the child process to finish
        printf("Parent process: Child finished. Exiting.\n");
    }

    return 0;
}
```

Output:


afiq@secr2043:~/os_lab2$ ./program
Starting the program...
Parent process (PID: 873): Waiting for child to finish...
Child process (PID: 874): Doing some work...
Child process: Finished work.
Parent process: Child finished. Exiting.